

Student:- karam mahameed, 213029143

Project Report on Phishing Detection Using Machine Learning Techniques

Source summary:

Phishing websites are designed to imitate legitimate websites in order to steal sensitive information such as usernames, passwords, and banking details. Since new phishing websites appear every day relying only on blacklists is often not enough. The paper examines whether machine learning can improve the detection of phishing websites by learning patterns from previously collected data.

The authors use a publicly available dataset that contains 11,055 website samples described by 30 features related to URLs, and webpage behavior. the use of an IP address instead of a domain name, HTTPS information, redirects, DNS records, page rank, and the age of the domain. Each sample is labeled as either phishing or legitimate.

To solve the problem, the paper compares several supervised machine learning algorithms, including Logistic Regression, Decision Tree, Random Forest and XGBoost. The models are evaluated using common classification metrics such as accuracy, precision, recall, and F1-score to determine which algorithm performs best on the dataset.

The main goal of the paper is to identify which machine learning approach is the most effective for phishing website detection while keeping the solution practical enough to be used in real-world cybersecurity systems.

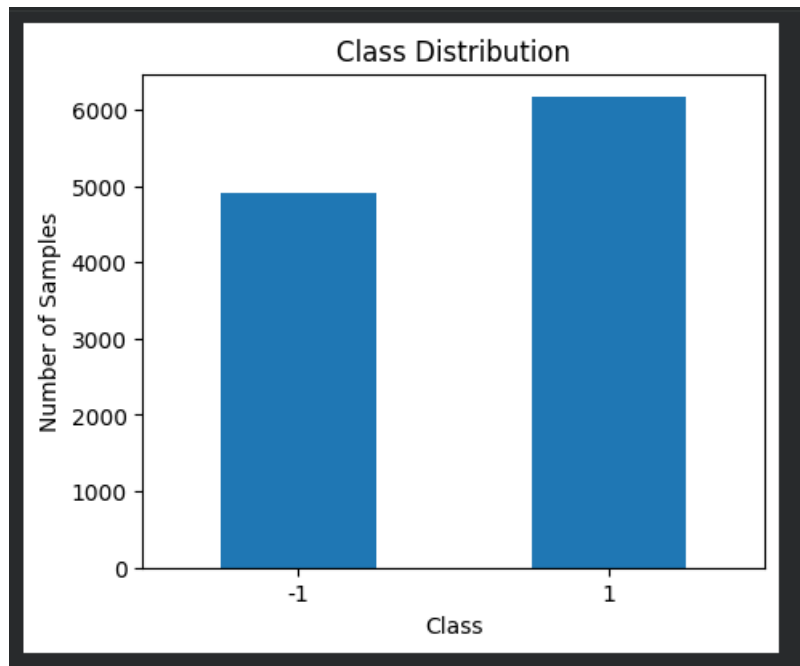
Evaluation:

The paper presents a clear comparison of several machine learning algorithms for phishing website detection and shows that ensemble methods achieve the best overall performance on the selected dataset. Based on both the results reported in the paper and my own reproduction, I generally agree with the authors' conclusion that tree based ensemble models perform better than simpler models such as Logistic Regression.

The evaluation methodology is appropriate for comparing different classifiers because every model is trained and tested on the same dataset using the same evaluation metrics. This makes the comparison fair and easy to understand. In my implementation, Random Forest also achieved the lowest number of classification errors, while XGBoost produced almost identical results. This is consistent with the paper's conclusion that ensemble methods are among the strongest performers.

	Model	Incorrect Predictions
0	Logistic Regression	158
1	Decision Tree	64
2	Random Forest	57
3	XGBoost	58

Although the reported results are convincing the study has several limitations. First, the dataset is almost balanced between phishing and legitimate websites while real-world environments are usually highly imbalanced. In practice, legitimate websites greatly outnumber phishing websites, so a model trained on a balanced dataset may achieve lower performance when deployed in a real system.



As we see above the -1 class represent the phishing websites and 1 represent the legitimate websites. Another limitation is the lack of temporal information. The dataset does not include the collection date of each website, making it impossible to evaluate how well the models perform against new phishing attacks. Since phishing techniques change over time, a time-based evaluation would provide a more realistic estimate of future performance.

```
*** Explicit temporal columns: []  
Time-related feature columns: ['Domain_Registration_Length', 'age_of_domain']
```

Overall, I believe the conclusions of the paper are justified. My reproduction produced similar behavior, with Random Forest and XGBoost outperforming the simpler models. However, the reported results should mainly be interpreted as performance on this specific dataset rather than as evidence that these models will always achieve the same level of success in real-world phishing detection.

Feature Engineering Analysis:

the dataset used in this project is already preprocessed so most of the feature engineering was completed by the authors before the data was published. Instead of using the original URLs directly each website is represented by 30 manually engineered features describing different characteristics of the URL the use of an IP address, HTTPS information, redirects, domain age, DNS records, page rank, and several HTML-related properties.

In my implementation The target variable was separated from the input features, and standardization was applied only for Logistic Regression because this model is sensitive to feature scales. Tree-based models such as Decision Tree, Random Forest, and XGBoost were trained using the original feature values since they are not affected by feature scaling.

During the exploratory data analysis, Spearman correlation was used to identify highly correlated features that could indicate redundancy. If several features carry nearly the same information, removing one of them could simplify the model without significantly affecting performance.

The feature engineering process is meaningful from both mathematical and cybersecurity perspectives. Mathematically, converting URLs and webpage properties into numerical features allows machine learning algorithms to learn decision boundaries between phishing and legitimate websites. the selected features describe behaviors that are commonly associated with phishing attacks.

Although the selected features provide good performance additional information could further improve the models. Since phishing techniques continue to evolve, regularly updating the feature set would likely improve the robustness of the system.

Reproducibility Analysis:

One of the strengths of this work is that the authors provide both the research paper and a public GitHub repository containing the implementation and the dataset. This made it possible to reproduce the experiments without having to rebuild the dataset from scratch. The repository also includes a list of the required Python packages, which made setting up the environment straightforward.

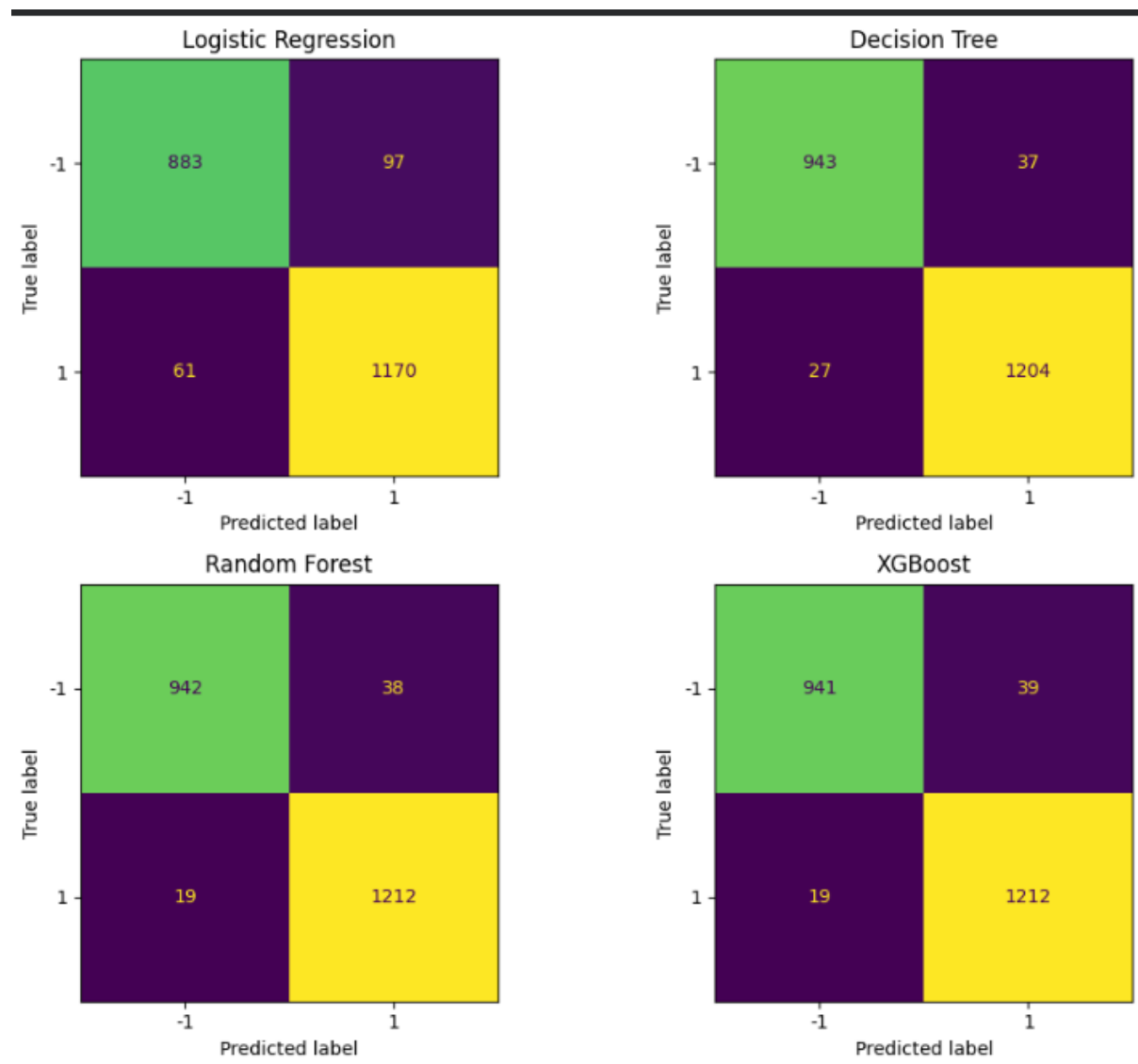
Although the project is mostly reproducible, some preprocessing steps are not fully documented. The released dataset already contains engineered features instead of the original URLs, so it is not possible to reproduce the complete feature extraction process from the raw data using only the provided CSV file. In addition, the paper does not clearly explain how websites with unavailable information, failed feature extraction, or inaccessible pages were handled during data collection.

Aside from all that, I consider the work to be highly reproducible. The paper, dataset, and source code are publicly available, allowing the main experiments to be repeated successfully. However, reproducing the entire data preparation pipeline would require additional documentation about the original preprocessing and feature extraction stages.

Experimental Results:

To evaluate the approach presented in the paper, I reproduced the phishing detection system using the dataset provided by the authors. Before training the models, the dataset was inspected, missing values were checked, irrelevant features were removed, and the data was divided into training and testing sets using an 80/20 stratified split.

Four machine learning models were trained and compared: Logistic Regression, Decision Tree, Random Forest, and XGBoost. These models were selected because they represent both simple linear methods and more advanced ensemble techniques. The goal was to compare their performance under the same experimental conditions.



The models were evaluated using Accuracy, Precision, Recall, F1-score, Matthews Correlation Coefficient (MCC), ROC-AUC, and the Confusion Matrix. These metrics provide a better understanding of model performance than accuracy alone. In phishing detection, false negatives are particularly important because they represent phishing websites that are incorrectly classified as legitimate.

	Model	Accuracy	Precision	Recall	F1	MCC
0	Logistic Regression	0.9285	0.9354	0.9010	0.9179	0.8551
1	Decision Tree	0.9711	0.9722	0.9622	0.9672	0.9413
2	Random Forest	0.9742	0.9802	0.9612	0.9706	0.9478
3	XGBoost	0.9738	0.9802	0.9602	0.9701	0.9469

The experimental results showed that Random Forest achieved the best overall performance in my implementation, producing the lowest number of classification errors. XGBoost achieved very similar results, differing by only one incorrect prediction. Decision Tree also performed well, while Logistic Regression produced the weakest performance among the evaluated models.

These results are generally consistent with the conclusions reported in the original paper. Both the paper and my implementation show that ensemble methods outperform simpler linear models for this phishing detection dataset. Although the exact performance values are not identical, the overall ranking of the models is similar, supporting the main claims of the authors.

Conclusions:

This project reproduced and evaluated the machine learning approach presented in the paper. The dataset was analyzed, explored, and used to train several classification models for phishing website detection.

The experimental results showed that ensemble methods performed better than the simpler models. Random Forest achieved the best overall performance in my implementation, while XGBoost produced very similar results. Logistic Regression achieved the lowest performance, suggesting that the relationships between the features and the target are more complex than a simple linear model can capture.

The main strength of the proposed solution is that it achieves high performance using relatively simple machine learning techniques and publicly available data. Another advantage is that both the paper and the implementation are publicly available, making the work easier to reproduce.

The main limitations are the balanced dataset, the lack of temporal information, and the limited description of the preprocessing steps used to create the final dataset. These factors make it difficult to estimate how well the models would perform on continuously evolving phishing attacks in real-world environments.

Future work could include testing the models on newer datasets, adding more informative features, and performing time-based validation to better simulate real deployment conditions.

Executive Summary:

This project presents a critical evaluation and reproduction of the paper Phishing Detection Using Machine Learning Techniques. The goal of the original work is to detect phishing websites using supervised machine learning based on engineered features extracted from URLs, domains, and webpage characteristics.

The experimental results showed that Random Forest achieved the best overall performance, with XGBoost producing very similar results. These findings are consistent with the main conclusions of the original paper, which reported that ensemble methods outperform simpler linear models on this dataset.

The critical evaluation identified several strengths, including the use of meaningful cybersecurity features, a public dataset, and an available GitHub repository that supports reproducibility. However, some limitations were also identified, such as the balanced class distribution, the lack of temporal information, and incomplete documentation of the preprocessing process.

Overall, the project demonstrates that machine learning can effectively detect phishing websites when suitable features are available. While the proposed approach is useful as a baseline solution, additional features and more realistic evaluation strategies would likely improve its performance in real-world cybersecurity applications.

References:

Paper :- V. Shahrivari and M. M. Darabi, "Phishing Detection Using Machine Learning Techniques," 2024.

GitHub Repository:- https://github.com/fafal-abnir/phishing_detection

UCI Machine Learning Repository. <https://archive.ics.uci.edu/>